How Quickly Does an Internet-Facing Server Get Scanned? An AWS EC2 Exposure Experiment

Name: Sumit Mhalaskar **PRN:** 2124UCSM1065 **Date of experiment:** [start date] to [end date]

> **How to use this report:** Fill every [bracketed value] with numbers from your own EC2 logs (commands are in Section 5). Do not submit invented numbers; the conclusion is only credible if it rests on your real evidence.

1. Objective

To understand how quickly an ordinary Internet-facing Linux server starts receiving automated scans and login attempts, and to test whether attackers really target only large companies.

2. Experimental Setup

Item	Value
Cloud provider	AWS EC2
Region	[e.g. ap-south-1 (Mumbai)]
Instance type	[e.g. t2.micro / t3.micro]
Operating system	[e.g. Amazon Linux 2023 / Ubuntu 22.04]
Launch time (UTC)	[date and time]
Log collection time (UTC)	[date and time]
Total uptime	[24 / 48 hours]
Public IP	Not shared with anyone and not published anywhere (kept private as required)

Security Group (inbound rules):

Type	Port	Source
SSH	22	0.0.0.0/0
HTTP	80	0.0.0.0/0

The server was a fresh, unremarkable machine: no domain name, no company association, no advertised service, and no data of value. Only the public IP existed.

3. Methodology

1. Launched a new EC2 Linux instance with the Security Group above.
2. Left it running untouched for [24/48] hours and shared the IP with nobody.
3. Logged in once, after the period, and analysed the SSH logs with journalctl and standard Linux text tools.
4. Recorded the number of failures, source IPs, usernames tried, and time to the first attempt.

4. Why this happens (background)

Internet-wide scanners such as masscan and zmap, along with botnets, can probe the entire IPv4 address space (about 4.3 billion addresses) in minutes to hours. Cloud provider address ranges are especially well known and are scanned continuously. A new instance receives a public IP from a known pool, so it is found without anyone knowing it exists. Attackers do not need to know who owns a server; they simply try every address.

5. Commands Used to Collect Evidence

Note: on Amazon Linux / RHEL the unit is `sshd`; on Ubuntu / Debian it is `ssh`, and the logs are also in `/var/log/auth.log`.

```
# Raw logs
sudo journalctl -u sshd --no-pager > ssh_all.log

# Failed attempts
sudo journalctl -u sshd | grep -i "failed" > ssh_failed.log

# Total number of failed attempts
```

```
grep -ci "failed password" ssh_failed.log

# Top source IPs
grep -i "failed password" ssh_failed.log | grep -oE "from [0-9.]+" | awk
'{print $2}' | sort | uniq -c | sort -nr | head -20

# Number of unique source IPs
grep -i "failed password" ssh_failed.log | grep -oE "from [0-9.]+" | awk
'{print $2}' | sort -u | wc -l

# Usernames attempted (valid users)
grep -i "failed password for" ssh_failed.log | grep -v "invalid user" | awk
'{print $(NF-5)}' | sort | uniq -c | sort -nr | head

# Usernames attempted (invalid users)
grep -i "invalid user" ssh_all.log | awk '{for(i=1;i<=NF;i++) if($i=="user")
print $(i+1)}' | sort | uniq -c | sort -nr | head -20

# First attempt timestamp
sudo journalctl -u sshd --no-pager | head -n 5

# Attempts per hour (to show automation / rhythm)
grep -i "failed password" ssh_failed.log | awk '{print $3}' | cut -d: -f1 |
sort | uniq -c
```

6. Results

6.1 Summary of findings

Question	Result from my logs
Total failed attempts	[number] in [24/48] hours
Unique source IPs	[number]
Time from launch to first attempt	[e.g. X minutes]
Most common username	[e.g. root] ([count] attempts)
Peak attack rate	[attempts per minute or hour]
Successful logins by attackers	[0] (key-based authentication only)

6.2 Top source IPs

Rank	Source IP	Attempts	Country / ASN (from whois or AbuseIPDB)
1	[ip]	[n]	[..]
2	[ip]	[n]	[..]
3	[ip]	[n]	[..]
4	[ip]	[n]	[..]
5	[ip]	[n]	[..]

6.3 Usernames attempted

Username	Attempts
root	[n]
admin	[n]
ubuntu	[n]
test / user / oracle / postgres / ftpuser etc.	[n]

(Paste the top 10 from your own output. Typical dictionary names include root, admin, ubuntu, ec2-user, test, user, oracle, postgres, git and pi.)

6.4 Time to first attempt

The instance was launched at [time]. The first failed or invalid SSH attempt was logged at [time], which is [X minutes] after launch.

6.5 Screenshots (attach)

- Security Group rules
- Output of the failed-attempts count
- Top source IPs
- Usernames list
- First log entry timestamp

7. Analysis: Were the Attempts Automated?

Evidence to check in your logs, and what each indicates:

- **Speed and volume:** Hundreds or thousands of attempts within hours, often several per second from one IP, is not humanly possible.
- **Dictionary usernames:** The same generic names (root, admin, ubuntu, test) appear across many IPs. A human targeting a specific company would try names related to it; here nothing relates to any organisation.
- **No targeting:** The server had no domain, no website content, and no link to any person or company, yet it was still attacked.
- **Distributed sources:** Many unrelated IPs (often from different countries and hosting providers) tried similar username lists, consistent with botnets and shared wordlists.
- **Repeated connect-fail-disconnect patterns:** Typical of scripts such as Hydra, Medusa or custom brute-force bots, with identical timing between tries.
- **Pre-authentication disconnects** ("Connection closed by … [preauth]"): Scanners that only fingerprint the SSH banner.

Conclusion for this question: Based on the above, the attempts were [almost certainly / very likely] automated.

8. Final Question: Do Attackers Really Target Only Big Companies?

No. The evidence from my EC2 instance supports this clearly.

1. **The first attempt arrived within [X minutes] of launch.** Nobody knew this server existed. It had no name, no customers and no reputation. The only way it could be found is by scanning every address, so size and fame play no role.
2. **[N] failed attempts from [M] unique IPs in [24/48] hours** on a machine holding no data of value shows that attacks are volume-based, not target-based.
3. **The usernames were generic** (root, admin, ubuntu and so on) rather than tailored to any organisation, so attackers were guessing, not choosing a victim.
4. **Attackers gain from any machine.** A compromised small server can be used as a botnet node, a crypto-miner, a spam or phishing relay, a proxy to hide larger attacks, or a stepping stone into a network. Small and poorly protected servers are in fact easier and more attractive targets than hardened corporate ones.

Therefore attackers do not primarily choose victims by size. Automated tools attack everything reachable, and the real risk factor is **exposure plus weak configuration**, not being a big company. Targeted attacks on large organisations do also exist, but they sit on top of this constant background of indiscriminate scanning, which every Internet-facing system, including a student's test server, faces from the first minutes online.

9. Recommendations / Mitigations

- Restrict SSH (22) to your own IP or use AWS Systems Manager Session Manager instead of open SSH.
- Disable password authentication and root login (`PasswordAuthentication no`, `PermitRootLogin no`) and use key pairs only.
- Install fail2ban or use rate limiting to block repeated failures.
- Change the SSH port only as a minor noise reducer, never as real security.
- Keep the OS patched and expose only the ports that are needed.
- Enable CloudWatch/GuardDuty logging and alerts.
- Use a bastion host or VPN for administrative access.

10. Cleanup and Ethics

After collecting logs, the instance was terminated to avoid cost and risk (and the Elastic/public IP released). The experiment only observed traffic arriving at my own server; no attempt was made to attack, scan or access any third-party system.

11. Conclusion

Within [X minutes] of going online, my otherwise anonymous EC2 server was being probed, and over [24/48] hours it recorded [N] failed login attempts from [M] IPs using common usernames such as [root, admin, ...]. This demonstrates that attacks are largely automated and indiscriminate. Attackers do not only target big companies; any Internet-exposed server will be found and tested almost immediately, so basic hardening is necessary for every system, regardless of size.

Submitted by: Sumit Mhalaskar | PRN: 2124UCSM1065